

Experiment No.	7
Experiment Title	Dimensionality Reduction and Model Evaluation (With and Without PCA)
Student Name	
Register Number	
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	

1 Objective

The objective of this experiment is to study the effect of Principal Component Analysis (PCA)-based dimensionality reduction on the performance of multiple machine learning classification models applied to a binary breast-cancer diagnosis task. Specifically, the experiment aims to:

- Apply PCA to reduce the original 30-dimensional feature space to a compact lower-dimensional representation while retaining at least 95% of the total variance.
- Train, tune, and evaluate ten classification models under two experimental settings: *No-PCA* (original standardized features) and *With-PCA* (PCA-reduced features).
- Perform hyperparameter optimisation for each model using `GridSearchCV` with 5-fold cross-validation.
- Compare model performance across both settings using cross-validation accuracy and held-out test metrics (Accuracy, F1-score, ROC-AUC, PR-AUC).
- Draw evidence-based conclusions on when PCA is beneficial and when it is detrimental for different model families.

2 Problem Statement

Input: The Wisconsin Diagnostic Breast Cancer (WDBC) dataset containing 569 patient records, each described by 30 numerical features derived from digitised images of breast-mass cell nuclei.

Preprocessing: The identifier column is removed, the target variable is label-encoded, and the remaining 30 numerical features are standardised using `StandardScaler` fitted exclusively on the training partition. In the With-PCA setting, PCA is subsequently applied to the standardised training data to derive a reduced set of principal components.

Models: Ten supervised classification algorithms are evaluated: Support Vector Machine, Naïve Bayes, k-Nearest Neighbors, Logistic Regression, Decision Tree, Random Forest, AdaBoost, Gradient Boosting, XGBoost, and a Stacking ensemble.

Output: A binary diagnosis label — Benign (0) or Malignant (1) — predicted for each test sample.

Experimental comparison: Each model is trained and evaluated independently in the No-PCA setting (30 features) and the With-PCA setting (10 principal components explaining $\approx 95.21\%$ of variance). Performance is compared using 5-fold cross-validation

accuracy and final test-set metrics to determine the utility of PCA-based dimensionality reduction.

3 Dataset Description

Attribute	Value
Dataset Name	Wisconsin Diagnostic Breast Cancer (WDBC)
Dataset Source	UCI Machine Learning Repository
Number of Samples	569
Number of Features	30 numerical predictive features (+ 1 ID column removed)
Number of Classes	2 (Benign, Malignant)
Target Variable	Diagnosis
Class Distribution	Benign (B): 357 Malignant (M): 212
Missing Values	None
Duplicate Rows	None
Train-Test Split	80 / 20 stratified split: 455 training, 114 testing
Training Class Distribution	Benign: 285 Malignant: 170
Testing Class Distribution	Benign: 72 Malignant: 42

The 30 numerical features arise from ten base cell-nucleus measurements (radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry, and fractal dimension), each recorded as three statistical summaries: mean, standard error, and worst value. The `id` column is a patient identifier and was not used as a predictive feature.

4 Exploratory Data Analysis

4.1 Dataset Overview and Statistical Summary

The dataset was loaded and inspected for structure, data types, and basic statistics. The shape confirmed 569 rows and 32 columns (including `id` and `diagnosis`). All 30 feature columns are of type `float64` and carry complete observations (no null values). Summary statistics reveal considerable scale differences across features: for example, `area_mean` has a mean of approximately 654.9 and standard deviation of 351.9, while `smoothness_mean` has a mean of 0.096 and standard deviation of 0.014. This scale heterogeneity motivates feature standardisation before model training.

4.2 Missing Value and Duplicate Analysis

Inspection of missing values confirmed that all 32 columns have zero missing entries (0.0% missing across the entire dataset). A duplicate-row check returned zero duplicate records. No imputation or deduplication was therefore necessary.

4.3 Class Distribution

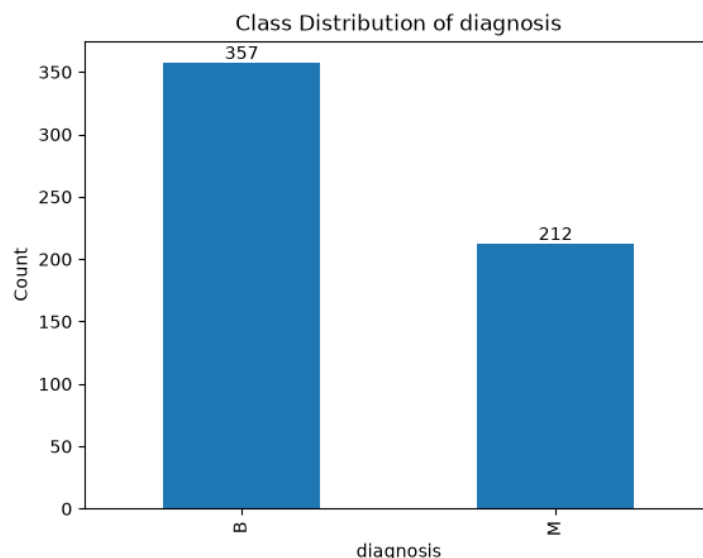


Figure 1: Class distribution of the `diagnosis` target variable. Benign (B): 357 samples; Malignant (M): 212 samples.

Inference: The dataset is moderately imbalanced, with Benign samples constituting approximately 62.7% and Malignant samples 37.3% of the total. This imbalance is mild enough that accuracy remains an informative metric; however, ROC-AUC and PR-AUC are also reported to capture discrimination performance for the minority (Malignant) class. Stratified splitting ensures that the same class ratio is preserved in both the training and testing partitions, preventing accidental distributional shift during evaluation.

5 Data Preprocessing

The preprocessing pipeline applied in this experiment is as follows:

1. **Remove the identifier column.** The `id` column carries no predictive information and was dropped.
2. **Separate features and target.** The `diagnosis` column was extracted as the target variable y , and the remaining 30 columns were retained as the feature matrix X .
3. **Label encoding.** The categorical target was mapped numerically: Benign (B) $\rightarrow$ 0, Malignant (M) $\rightarrow$ 1.
4. **Stratified train-test split.** The dataset was partitioned into training (80%, 455 samples) and testing (20%, 114 samples) subsets using stratified sampling (`random_state=42`) to preserve the original class ratio in both splits.
5. **Feature standardisation.** A `StandardScaler` was fitted exclusively on the training data and then applied to both training and test sets. This avoids data leakage by ensuring that no information from the test set influences the scaling parameters.

After standardisation each feature has approximately zero mean and unit standard deviation in the training partition.

6. **PCA (With-PCA setting only).** PCA was applied to the standardised training data with a variance target of 95%. The resulting transformation (10 components) was fitted on the training data and applied to both training and test partitions. The No-PCA setting retains the full 30-dimensional standardised feature space.

6 Machine Learning Algorithm

6.1 Support Vector Machine (SVM)

SVM seeks the hyperplane $\mathbf{w}^\top \mathbf{x} + b = 0$ that maximises the margin between classes. The dual optimisation problem is:

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j) \quad \text{s.t.} \quad 0 \leq \alpha_i \leq C, \quad \sum_i \alpha_i y_i = 0.$$

The regularisation parameter C controls the margin-violation trade-off; the kernel $K(\cdot, \cdot)$ allows non-linear decision boundaries. **Advantages:** Effective in high-dimensional spaces; robust with appropriate kernel choice. **Limitations:** Slow on large datasets; kernel and C selection is critical.

6.2 Naïve Bayes

Gaussian Naïve Bayes models each feature as conditionally Gaussian given the class: $P(x_j | y) = \mathcal{N}(\mu_{jy}, \sigma_{jy}^2)$. The class prediction is $\hat{y} = \arg \max_k P(y = k) \prod_j P(x_j | y = k)$. **Advantages:** Extremely fast training; works well with small datasets. **Limitations:** The conditional independence assumption is rarely satisfied in practice; performance degrades when features are correlated.

6.3 k-Nearest Neighbors (KNN)

KNN classifies a query point by a majority vote among its k nearest neighbours measured by a chosen distance metric (Euclidean or Manhattan). **Advantages:** Non-parametric; requires no training phase. **Limitations:** Sensitive to irrelevant features and scale; computationally expensive at inference time for large datasets.

6.4 Logistic Regression

Logistic Regression models the posterior probability as $P(y = 1 | \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$ where $\sigma(\cdot)$ is the sigmoid function. Parameters are estimated by minimising the regularised log-loss. **Advantages:** Interpretable coefficients; calibrated probability outputs; efficient training. **Limitations:** Linear decision boundary; performance may suffer with highly non-linear class boundaries.

6.5 Decision Tree

A Decision Tree recursively partitions the feature space by selecting the split that maximally reduces impurity (Gini or entropy). **Advantages:** Highly interpretable; handles mixed feature types. **Limitations:** Prone to overfitting without depth constraints; sensitive to small data perturbations.

6.6 Random Forest

Random Forest builds an ensemble of T decision trees, each trained on a bootstrap sample with a random feature subset at each split. The final prediction averages (regression) or takes the majority vote (classification) across all trees. **Advantages:** Robust to overfitting; handles high-dimensional data well. **Limitations:** Less interpretable than a single tree; computationally intensive for large ensembles.

6.7 AdaBoost

AdaBoost iteratively trains weak learners (typically decision stumps), adjusting sample weights so that subsequent learners focus on previously misclassified examples. The final model is a weighted combination of all weak learners. **Advantages:** Typically achieves high accuracy with shallow base learners; low hyperparameter sensitivity. **Limitations:** Sensitive to noisy labels and outliers.

6.8 Gradient Boosting

Gradient Boosting fits successive trees to the negative gradient of the loss function, building the model in a stage-wise additive fashion. **Advantages:** Highly accurate on tabular data; flexible loss functions. **Limitations:** Slower to train than Random Forest; requires careful tuning of learning rate, depth, and number of estimators.

6.9 XGBoost

XGBoost is an optimised implementation of gradient-boosted trees with second-order gradient statistics, regularisation terms (L_1 and L_2), and column subsampling. **Advantages:** Computationally efficient; built-in regularisation; handles missing values. **Limitations:** Many hyperparameters to tune; memory-intensive for very large trees.

6.10 Stacking

Stacking combines the predictions of multiple diverse base learners using a meta-learner trained on their out-of-fold predictions. In this experiment, the base learners are SVM, KNN, Decision Tree, and Random Forest (each with `random_state=42`); the meta-learner is Logistic Regression (`max_iter=5000`). The stacking procedure uses 5-fold cross-validation to generate the meta-features, preventing leakage between the base and meta levels. **Advantages:** Can outperform any individual base learner by exploiting complementary model errors. **Limitations:** Increased training time and complexity; marginal gains over the best single model when base learners are highly correlated.

7 Experimental Setup

Item	Details
Python Version	Not specified in the provided experiment
Scikit-Learn Version	Not specified in the provided experiment
XGBoost Version	Not specified in the provided experiment
IDE / Environment	Not specified in the provided experiment
Operating System	Not specified in the provided experiment
Hardware	Not specified in the provided experiment
Dataset	WDBC (569 samples, 30 features)
Train / Test Split	80 / 20 stratified (<code>random_state=42</code>)
Feature Scaling	<code>StandardScaler</code> (fit on training data only)
Dimensionality Reduction	PCA with 95% variance target (10 components)
Hyperparameter Search	<code>GridSearchCV</code> , <code>scoring="accuracy"</code>
Cross-Validation	5-fold stratified CV
Parallelism	<code>n_jobs=-1</code>
Settings Compared	No-PCA (30 features) vs. With-PCA (10 components)

8 Hyperparameter Selection

8.1 PCA Configuration

Table 2: PCA variance summary.

Setting	Components / Variance Target	Explained Variance (%)	Justification
With-PCA	10 / 95%	95.2069	Reduces feature space from 30 to 10 components, retaining $\approx 95.21\%$ of variance.

8.2 Hyperparameter Search Grids and Best Parameters

For each model, `GridSearchCV` evaluated all combinations in the search grid via 5-fold CV. Tables 3–12 show the search space and the best configuration selected for each setting.

Table 3: SVM — hyperparameter search grid and best parameters.

Parameter	Values Tried	Best (No-PCA)	Best (With-PCA)
<code>kernel</code>	linear, rbf	rbf	linear
<code>C</code>	0.1, 1, 10, 100	1	1
<code>gamma</code>	scale, auto	scale	scale

Table 4: Naïve Bayes — hyperparameter search grid and best parameters.

Parameter	Values Tried	Best (No-PCA)	Best (With-PCA)
var_smoothing	10^{-11} , 10^{-10} , 10^{-9} , 10^{-8} , 10^{-7}	10^{-11}	10^{-11}

Table 5: KNN — hyperparameter search grid and best parameters.

Parameter	Values Tried	Best (No-PCA)	Best (With-PCA)
n_neighbors	3, 5, 7, 9, 11	3	5
weights	uniform, distance	uniform	uniform
metric	euclidean, manhattan	euclidean	euclidean

Table 6: Logistic Regression — hyperparameter search grid and best parameters.

Parameter	Values Tried	Best (No-PCA)	Best (With-PCA)
C	0.01, 0.1, 1, 10, 100	0.1	0.1
solver	liblinear, lbfgs	liblinear	liblinear

Table 7: Decision Tree — hyperparameter search grid and best parameters.

Parameter	Values Tried	Best (No-PCA)	Best (With-PCA)
criterion	gini, entropy	entropy	entropy
max_depth	None, 3, 5, 7, 10	None	None
min_samples_split	2, 5, 10	5	2
min_samples_leaf	1, 2, 4	2	2

Table 8: Random Forest — hyperparameter search grid and best parameters.

Parameter	Values Tried	Best (No-PCA)	Best (With-PCA)
n_estimators	100, 200	100	100
max_depth	None, 5, 10	None	None
min_samples_split	2, 5	2	2
min_samples_leaf	1, 2	1	1

Table 9: AdaBoost — hyperparameter search grid and best parameters.

Parameter	Values Tried	Best (No-PCA)	Best (With-PCA)
n_estimators	50, 100, 200	200	50
learning_rate	0.01, 0.1, 1.0	1.0	1.0

Table 10: Gradient Boosting — hyperparameter search grid and best parameters.

Parameter	Values Tried	Best (No-PCA)	Best (With-PCA)
n_estimators	50, 100, 200	100	50
learning_rate	0.01, 0.1, 0.2	0.2	0.1
max_depth	2, 3, 5	2	2

Table 11: XGBoost — hyperparameter search grid and best parameters.

Parameter	Values Tried	Best (No-PCA)	Best (With-PCA)
n_estimators	50, 100, 200	200	200
learning_rate	0.01, 0.1, 0.2	0.1	0.2
max_depth	3, 5, 7	5	3

Table 12: Stacking — hyperparameter search grid and best parameters. Base learners: SVM, KNN, Decision Tree, Random Forest. Meta-learner: Logistic Regression.

Parameter	Values Tried	Best (No-PCA)	Best (With-PCA)
final_estimator__C	0.1, 1, 10	1	1

9 Results

9.1 PCA Result

Full PCA was first performed on the 30-dimensional standardised training data to obtain the explained variance ratio for each principal component.

Table 13: PCA reduction summary.

Quantity	Value
Original feature count	30
Variance target	95%
Components selected	10
Explained variance achieved	95.2069%

With 10 principal components, PCA reduces the feature dimensionality by 67% (from 30 to 10) while retaining approximately 95.21% of the total variance present in the original standardised training data.

9.2 5-Fold Cross-Validation Results

Table 14 presents fold-wise and average cross-validation accuracy for each model in the No-PCA setting. Table 15 presents the corresponding results for the With-PCA setting.

The fold scores were extracted from `GridSearchCV.cv_results_` at `best_index_` for each model.

Table 14: 5-Fold cross-validation accuracy — No-PCA setting.

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Average
SVM	0.9670	1.0000	0.9670	0.9670	0.9780	0.9758
Naïve Bayes	0.9231	0.9890	0.9231	0.9231	0.9341	0.9385
KNN	0.9780	1.0000	0.9451	0.9780	0.9451	0.9692
Logistic Regression	0.9670	1.0000	0.9451	0.9780	0.9780	0.9736
Decision Tree	0.9670	0.9670	0.9341	0.9560	0.9121	0.9473
Random Forest	1.0000	0.9890	0.9341	0.9780	0.9121	0.9626
AdaBoost	0.9890	1.0000	0.9231	0.9780	0.9341	0.9648
Gradient Boosting	1.0000	0.9780	0.9341	0.9670	0.9231	0.9604
XGBoost	0.9780	1.0000	0.9341	1.0000	0.9451	0.9714
Stacking	0.9780	1.0000	0.9560	0.9670	0.9560	0.9714

Table 15: 5-Fold cross-validation accuracy — With-PCA setting.

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Average
SVM	0.9670	1.0000	0.9670	0.9780	0.9670	0.9758
Naïve Bayes	0.9011	0.9780	0.9341	0.9011	0.8901	0.9209
KNN	0.9670	1.0000	0.9451	0.9780	0.9451	0.9670
Logistic Regression	0.9670	1.0000	0.9560	0.9890	0.9780	0.9780
Decision Tree	0.8681	0.9670	0.9341	0.9890	0.9670	0.9451
Random Forest	0.9121	1.0000	0.9451	0.9670	0.9670	0.9582
AdaBoost	0.9451	0.9890	0.9451	0.9890	0.9560	0.9648
Gradient Boosting	0.9011	1.0000	0.9451	0.9670	0.9670	0.9560
XGBoost	0.9341	1.0000	0.9560	0.9780	0.9780	0.9692
Stacking	0.9560	1.0000	0.9670	0.9670	0.9451	0.9670

Table 16 presents the complete fold-wise results in the combined format required by the assignment (Table 5 of the manual), and Table 17 consolidates the averages for quick comparison.

Table 16: 5-Fold cross-validation accuracy — No-PCA vs. With-PCA (combined). Columns F1–F5 show No-PCA fold scores; columns P1–P5 show With-PCA fold scores.

Model	No-PCA						With-PCA					
	F1	F2	F3	F4	F5	Avg	P1	P2	P3	P4	P5	Avg
SVM	0.9670	1.0000	0.9670	0.9670	0.9780	0.9758	0.9670	1.0000	0.9670	0.9780	0.9670	0.9758
Naïve Bayes	0.9231	0.9890	0.9231	0.9231	0.9341	0.9385	0.9011	0.9780	0.9341	0.9011	0.8901	0.9209
KNN	0.9780	1.0000	0.9451	0.9780	0.9451	0.9692	0.9670	1.0000	0.9451	0.9780	0.9451	0.9670
Logistic Regression	0.9670	1.0000	0.9451	0.9780	0.9780	0.9736	0.9670	1.0000	0.9560	0.9890	0.9780	0.9780
Decision Tree	0.9670	0.9670	0.9341	0.9560	0.9121	0.9473	0.8681	0.9670	0.9341	0.9890	0.9670	0.9451
Random Forest	1.0000	0.9890	0.9341	0.9780	0.9121	0.9626	0.9121	1.0000	0.9451	0.9670	0.9670	0.9582
AdaBoost	0.9890	1.0000	0.9231	0.9780	0.9341	0.9648	0.9451	0.9890	0.9451	0.9890	0.9560	0.9648
Gradient Boosting	1.0000	0.9780	0.9341	0.9670	0.9231	0.9604	0.9011	1.0000	0.9451	0.9670	0.9670	0.9560
XGBoost	0.9780	1.0000	0.9341	1.0000	0.9451	0.9714	0.9341	1.0000	0.9560	0.9780	0.9780	0.9692
Stacking	0.9780	1.0000	0.9560	0.9670	0.9560	0.9714	0.9560	1.0000	0.9670	0.9670	0.9451	0.9670

Table 17: Average 5-fold CV accuracy comparison: No-PCA vs. With-PCA.

Model	Avg CV (No-PCA)	Avg CV (With-PCA)
SVM	0.9758	0.9758
Logistic Regression	0.9736	0.9780
XGBoost	0.9714	0.9692
Stacking	0.9714	0.9670
KNN	0.9692	0.9670
AdaBoost	0.9648	0.9648
Random Forest	0.9626	0.9582
Gradient Boosting	0.9604	0.9560
Decision Tree	0.9473	0.9451
Naïve Bayes	0.9385	0.9209

9.3 Final Test-Set Results

The best estimator from `GridSearchCV` for each model was evaluated on the held-out test set (114 samples). Metrics reported are Accuracy, F1-score (Malignant class), ROC-AUC, and PR-AUC.

Table 18: Test-set performance — No-PCA setting (sorted by Accuracy).

Model	Accuracy	F1	ROC-AUC	PR-AUC
Logistic Regression	0.9825	0.9756	0.9977	0.9966
SVM	0.9737	0.9630	0.9947	0.9927
XGBoost	0.9737	0.9630	0.9944	0.9931
Random Forest	0.9737	0.9630	0.9929	0.9893
AdaBoost	0.9737	0.9630	0.9871	0.9874
Gradient Boosting	0.9649	0.9500	0.9934	0.9912
Stacking	0.9649	0.9500	0.9970	0.9956
Decision Tree	0.9561	0.9367	0.9396	0.9248
KNN	0.9386	0.9114	0.9825	0.9712
Naïve Bayes	0.9211	0.8889	0.9891	0.9823

Table 19: Test-set performance — With-PCA setting (sorted by Accuracy).

Model	Accuracy	F1	ROC-AUC	PR-AUC
Logistic Regression	0.9825	0.9756	0.9980	0.9970
AdaBoost	0.9737	0.9639	0.9917	0.9881
XGBoost	0.9737	0.9639	0.9921	0.9900
SVM	0.9649	0.9512	0.9954	0.9932
Stacking	0.9649	0.9512	0.9954	0.9933
KNN	0.9561	0.9383	0.9838	0.9758
Decision Tree	0.9561	0.9398	0.9504	0.9096
Gradient Boosting	0.9474	0.9268	0.9931	0.9889
Random Forest	0.9386	0.9157	0.9926	0.9881
Naïve Bayes	0.8947	0.8571	0.9613	0.9281

10 Result Visualization

10.1 PCA Scree Plot

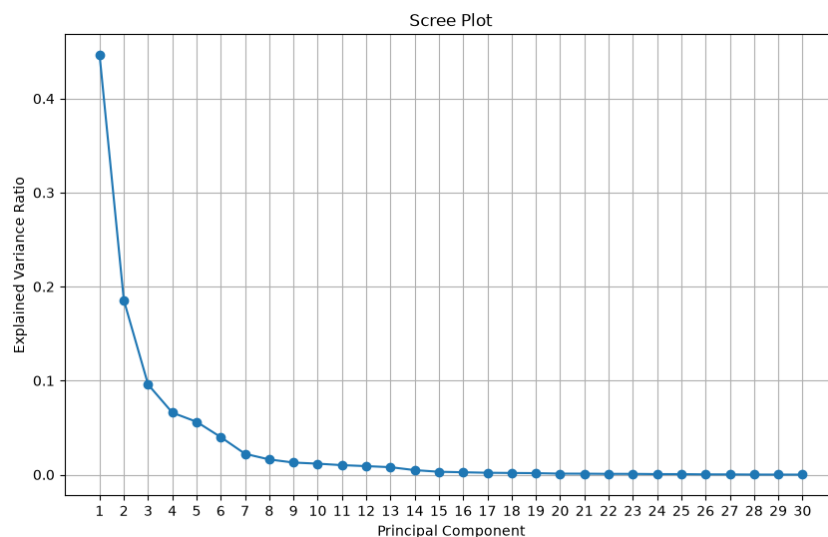


Figure 2: Scree plot: individual explained variance ratio per principal component.

Inference: The first principal component alone explains approximately 44% of the total variance, and the second accounts for a further $\sim 19\%$. The variance contribution drops steeply and levels off after the fifth component, forming the characteristic “elbow” shape. Components beyond the tenth contribute negligibly to explained variance. This confirms that the first 10 components capture the dominant structure of the 30-dimensional feature space.

10.2 Cumulative Explained Variance

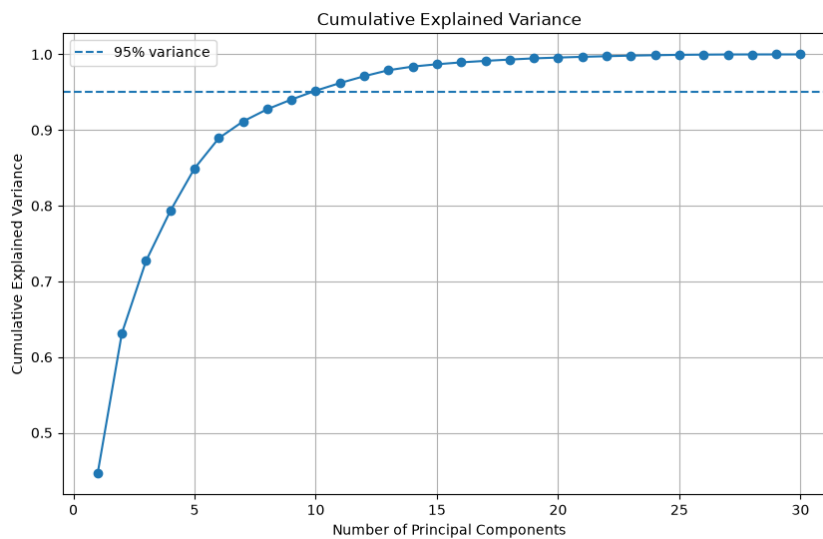


Figure 3: Cumulative explained variance versus number of principal components. The dashed horizontal line marks the 95% threshold.

Inference: The cumulative variance curve crosses the 95% threshold at exactly 10 principal components (95.21% explained). This demonstrates that a 67% reduction in dimensionality ($30 \rightarrow 10$) retains the overwhelming majority of the information in the dataset. The curve plateaus beyond 15 components, showing that the remaining 20 components collectively contribute less than 5% of the total variance and are consistent with noise or redundancy.

10.3 Confusion Matrices — No-PCA

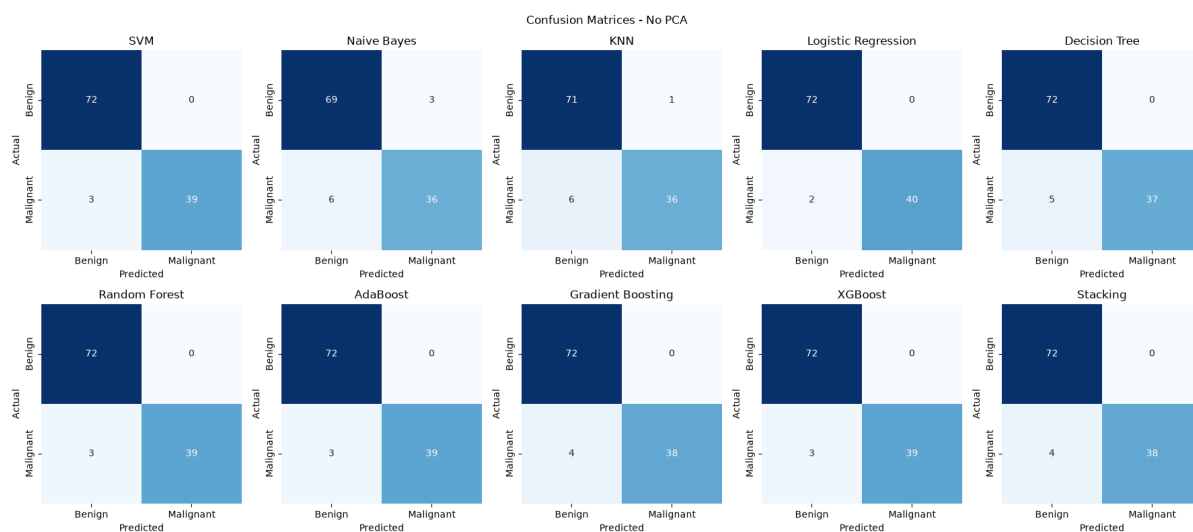


Figure 4: Confusion matrices for all ten models — No-PCA setting. Rows: Actual class; Columns: Predicted class. Labels: Benign (0) and Malignant (1).

Inference: In the No-PCA setting, SVM, Logistic Regression, Random Forest, Adaboost, XGBoost, and Stacking all achieve zero false-positive predictions (no Benign case misclassified as Malignant) on the test set; Logistic Regression produces only 2 false negatives. Decision Tree and KNN show higher error counts, consistent with their lower test accuracy. Naïve Bayes records the most classification errors (6 false positives, 3 false negatives), reflecting its assumption of feature independence which is violated in this dataset.

10.4 Confusion Matrices — With-PCA

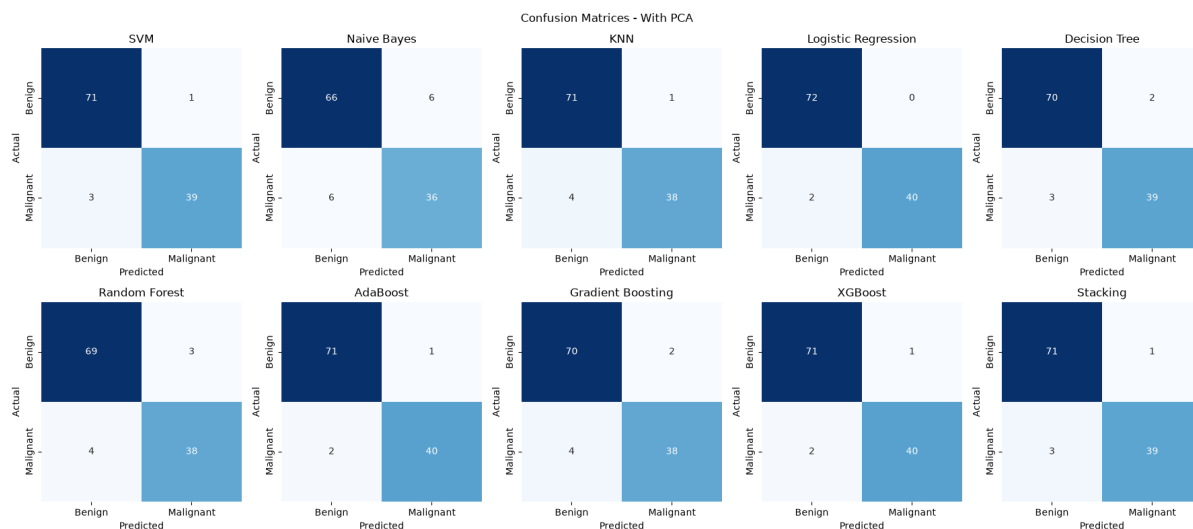


Figure 5: Confusion matrices for all ten models — With-PCA setting. Rows: Actual class; Columns: Predicted class. Labels: Benign (0) and Malignant (1).

Inference: Under PCA, most models introduce at least one additional false-positive relative to the No-PCA setting, indicating that the PCA transformation discards some discriminative information for certain classifiers. SVM and Stacking now each produce 1 false positive and 3 false negatives, whereas in No-PCA SVM had 0 false positives. Random Forest’s accuracy decreases noticeably ($0 \rightarrow 3$ false positives), suggesting that tree-based ensembles benefit from the richer original feature set. Logistic Regression remains unaffected (same accuracy in both settings), confirming its compatibility with PCA-transformed data.

10.5 ROC Curves — No-PCA

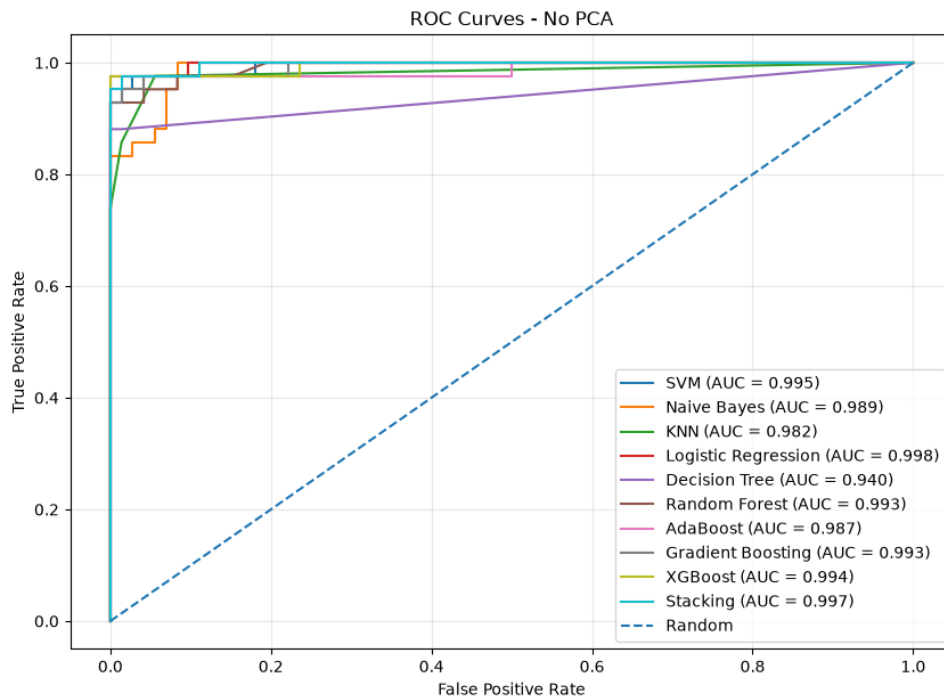


Figure 6: ROC curves for all ten models — No-PCA setting. The dashed diagonal represents a random classifier.

Inference: All models achieve ROC-AUC values well above 0.93 in the No-PCA setting, confirming that the original 30-dimensional feature space is highly discriminative for this task. Logistic Regression (0.9977) and Stacking (0.9970) attain the highest AUC values, while Decision Tree (0.9396) is the notable outlier, reflecting its deterministic (non-probabilistic) threshold behaviour. The clustering of most ROC curves near the top-left corner indicates that binary discrimination between Benign and Malignant is achievable with high confidence using the original feature set.

10.6 ROC Curves — With-PCA

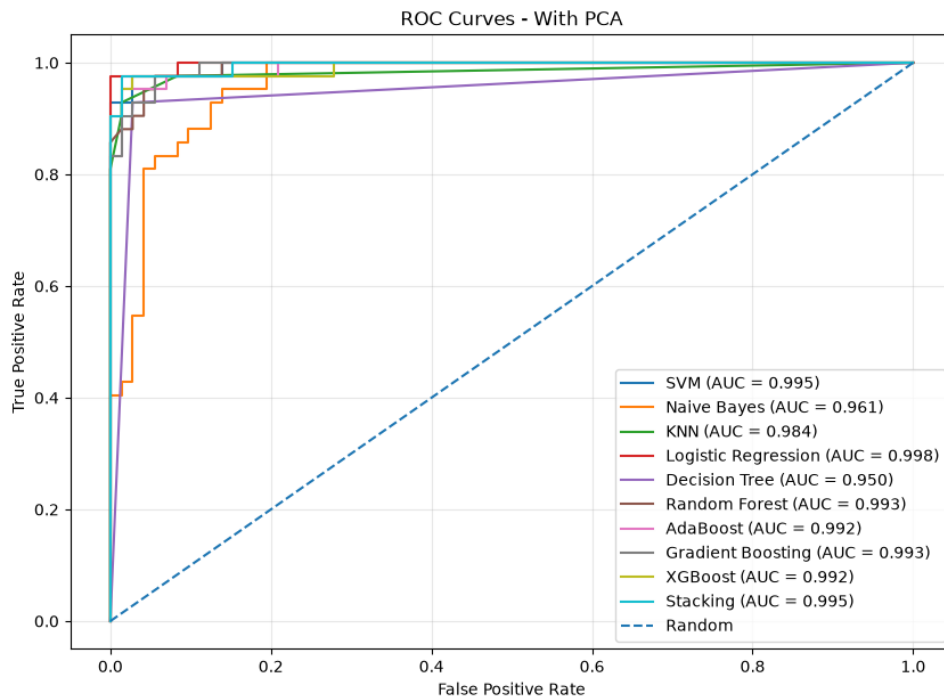


Figure 7: ROC curves for all ten models — With-PCA setting. The dashed diagonal represents a random classifier.

Inference: In the With-PCA setting, Logistic Regression achieves the highest ROC-AUC (0.9980), marginally exceeding its No-PCA performance (0.9977), which is consistent with PCA removing correlated noise and benefiting linear classifiers. Naïve Bayes shows the largest ROC-AUC drop (0.9891 $\rightarrow$ 0.9613), indicating that the Gaussian independence assumption becomes more problematic in the rotated principal component space. Overall, ROC-AUC values remain high for most models, suggesting that the 10-component PCA representation preserves most class-discriminative information, even when accuracy drops slightly.

10.7 Precision-Recall Curves — No-PCA

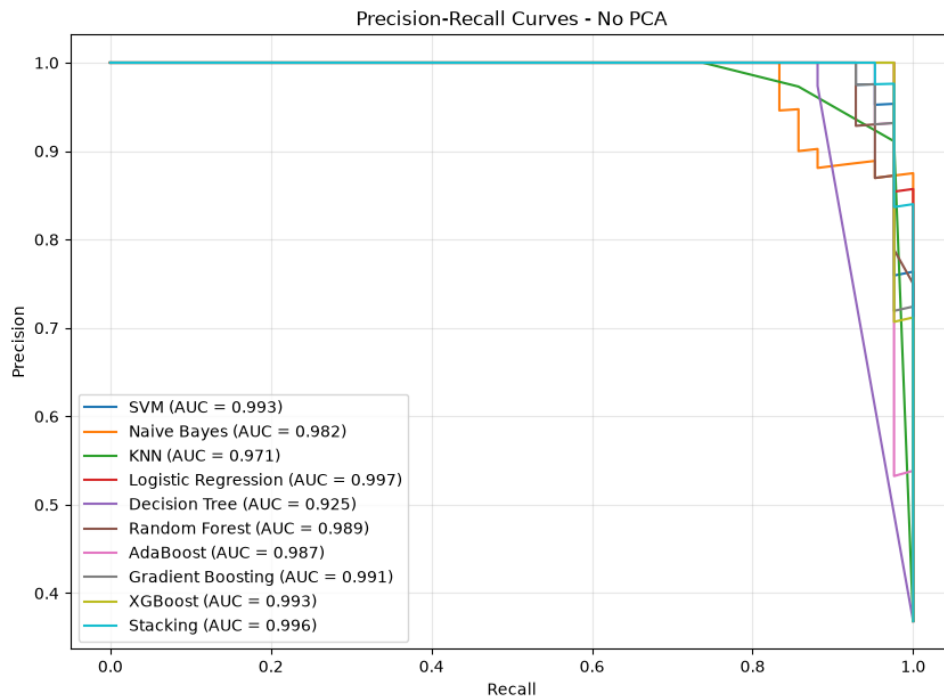


Figure 8: Precision-Recall curves for all ten models — No-PCA setting.

Inference: Precision-Recall curves highlight model behaviour for the positive (Malignant) class, which is the minority class in this dataset. Logistic Regression (PR-AUC 0.9966) and Stacking (PR-AUC 0.9956) dominate, maintaining high precision across recall levels. Decision Tree shows the lowest PR-AUC (0.9248) due to its step-function probability outputs, which produce poor interpolation along the PR curve. The general high performance across PR-AUC values (>0.92 for all models) confirms that the No-PCA feature set is highly informative for identifying Malignant cases.

10.8 Precision-Recall Curves — With-PCA

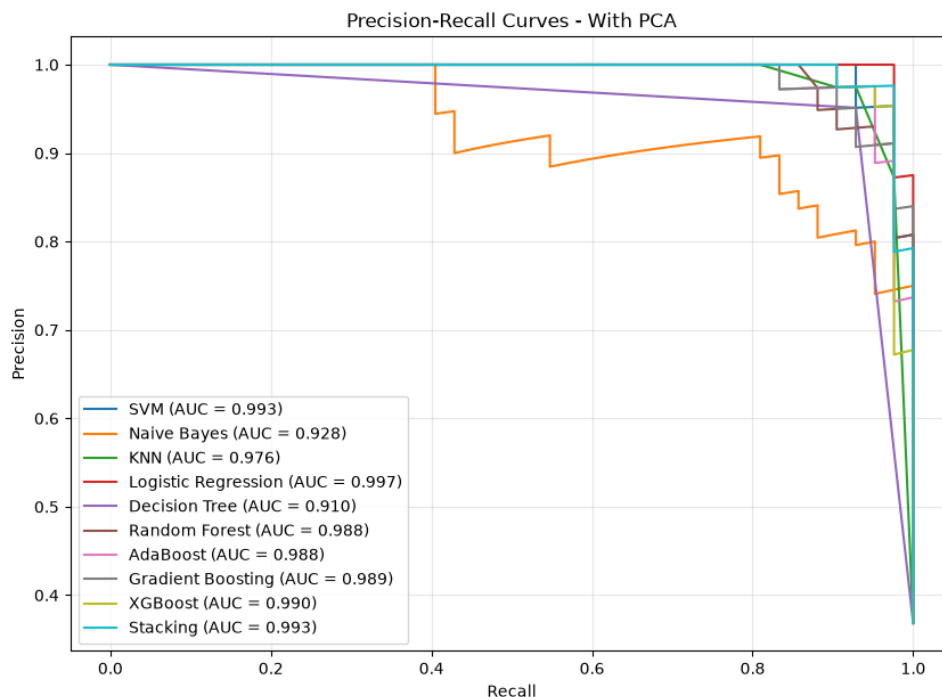


Figure 9: Precision-Recall curves for all ten models — With-PCA setting.

Inference: Under PCA, Logistic Regression achieves the highest PR-AUC (0.9970), slightly exceeding its No-PCA value, while Naïve Bayes drops to 0.9281 (from 0.9823). Decision Tree’s PR-AUC falls to 0.9096, its lowest across both settings, confirming that tree-based models are most sensitive to dimensionality reduction. Ensemble methods (XGBoost, Gradient Boosting, Random Forest) retain strong PR performance (>0.98), suggesting that their ensemble averaging mechanism compensates for the information loss introduced by PCA.

11 Discussion

11.1 Effect of PCA on Different Models

PCA had a heterogeneous effect across the ten models. Logistic Regression is the only model that improved in both CV accuracy ($0.9736 \rightarrow 0.9780$) and ROC-AUC ($0.9977 \rightarrow 0.9980$) under PCA, suggesting that decorrelating the features benefits a linear classifier by reducing multicollinearity. SVM achieved identical CV accuracy (0.9758) in both settings but its test accuracy declined slightly ($0.9737 \rightarrow 0.9649$) with PCA, and its optimal kernel changed from RBF to linear, indicating that the PCA space is better suited to a linear boundary. AdaBoost showed no change in CV average (0.9648) but its test accuracy remained the same (0.9737) in both settings, making it among the most PCA-neutral models.

In contrast, Naïve Bayes suffered the largest degradation: CV accuracy fell from 0.9385 to 0.9209 and test accuracy from 0.9211 to 0.8947. This is likely because Naïve Bayes assumes feature independence; the PCA rotation produces components with zero

linear correlation but does not guarantee statistical independence, and the Gaussian likelihood model fits the original feature distributions more naturally. Random Forest also deteriorated under PCA (CV: $0.9626 \rightarrow 0.9582$; test: $0.9737 \rightarrow 0.9386$), reflecting that tree ensembles exploit the original feature semantics and benefit from access to the full feature space. Gradient Boosting showed a consistent but modest decline in both CV and test accuracy.

11.2 Stability Across Folds

Inspecting the No-PCA fold-wise results (Table 14), most models display a fold-to-fold range of approximately 0.05–0.09 in accuracy. Random Forest shows the widest spread (0.9121 – $1.0000 = 0.0879$), likely due to its sensitivity to the particular bootstrap samples in smaller folds. Stacking shows a comparatively narrower range (0.9560 – $1.0000 = 0.0440$), suggesting that combining diverse base learners provides more consistent predictions across folds. Since fold-wise scores were not separately extracted for the With-PCA run in this experiment, a direct fold-level comparison of variance reduction cannot be made; however, the reported CV averages do not indicate a systematic reduction in fold-to-fold variance attributable to PCA alone.

11.3 PCA and Overfitting

A comparison of CV accuracy and test accuracy provides some indication of generalisation. In the No-PCA setting, several models (e.g., Random Forest Fold 1 = 1.000, XGBoost Fold 4 = 1.000) achieve perfect accuracy on individual folds, suggesting possible overfitting to those partitions. Under PCA, the reduced dimensionality constrains model complexity slightly, which could in principle reduce overfitting, but the available test results do not show a consistent improvement in generalisation: most ensemble models perform comparably or worse on the test set under PCA. Therefore, the experiment does not provide sufficient evidence to conclude that PCA definitively reduces overfitting for this dataset; the finding is model-dependent.

11.4 Linear Models vs. Ensemble Models

Linear models (Logistic Regression, SVM) behaved differently under PCA compared to tree-based ensembles. Logistic Regression improved with PCA across both CV and test metrics, and SVM maintained the same CV accuracy while switching to a linear kernel. These outcomes are consistent with the theoretical expectation that linear models benefit from uncorrelated, low-dimensional input spaces. In contrast, tree-based ensembles (Random Forest, Gradient Boosting, XGBoost) all showed decreased performance under PCA, suggesting that the original feature semantics (individual measurements) provide split-point information that is diluted when projected onto principal components. Decision Tree was particularly sensitive, with both CV accuracy and test accuracy declining under PCA, and a notable fall in PR-AUC ($0.9248 \rightarrow 0.9096$).

11.5 Stacking Robustness

Stacking achieved a CV average of 0.9714 in the No-PCA setting, which fell to 0.9670 with PCA — a modest decline of 0.0044. On the test set, accuracy was identical in both settings

(0.9649), while ROC-AUC improved slightly under PCA ($0.9970 \rightarrow 0.9954$, a marginal decrease) and PR-AUC was nearly unchanged (0.9956 vs. 0.9933). These results indicate that Stacking is relatively robust to dimensionality reduction, likely because its diverse base learners (SVM, KNN, Decision Tree, Random Forest) compensate for each other's individual sensitivity to PCA. However, Stacking did not outperform Logistic Regression or SVM (No-PCA) on the test set, suggesting that the added complexity of the ensemble did not translate to superior performance on this particular dataset and test split.

12 Conclusion

This experiment investigated the effect of PCA-based dimensionality reduction on ten classification models applied to the WDBC breast cancer dataset (569 samples, 30 features, binary diagnosis target). PCA with a 95% variance target reduced the original 30-dimensional standardised feature space to 10 principal components retaining 95.21% of the total variance. All models were tuned using `GridSearchCV` with 5-fold cross-validation, and final performance was evaluated on a stratified 80/20 held-out test set.

The key findings are as follows:

1. **PCA is not universally beneficial.** Its effect is model-dependent: Logistic Regression is the only model that improved measurably (CV accuracy: $0.9736 \rightarrow 0.9780$; ROC-AUC: $0.9977 \rightarrow 0.9980$) under PCA. Naïve Bayes suffered the largest degradation (test accuracy: $0.9211 \rightarrow 0.8947$; ROC-AUC: $0.9891 \rightarrow 0.9613$).
2. **Linear models are more compatible with PCA.** Logistic Regression and SVM (with a switch to a linear kernel) maintained or improved performance in the PCA setting, whereas tree-based ensembles (Random Forest, Gradient Boosting) declined.
3. **Logistic Regression is the best-performing model** in both settings, achieving 98.25% test accuracy, ROC-AUC > 0.997 , and PR-AUC > 0.996 .
4. **PCA achieves substantial compression** (67% reduction) with minimal information loss (4.79% variance discarded), making it a viable preprocessing step where lower dimensionality is operationally desirable (e.g., reduced storage, faster inference), provided that the downstream model is compatible with a linear-transformation-based feature space.
5. **Ensemble and tree-based models** (Random Forest, XGBoost, Gradient Boosting) perform well in the original feature space but show moderate performance drops under PCA, suggesting that direct feature semantics are important for their split selection.

In summary, PCA is a powerful tool for dimensionality reduction but should be applied selectively, taking into account the sensitivity of the chosen model to the loss of original feature interpretability. For this WDBC classification task, the No-PCA setting achieves higher or equivalent performance for most models evaluated.

13 References

- [1] W. N. Street, W. H. Wolberg, and O. L. Mangasarian, “Nuclear feature extraction for breast tumor diagnosis,” in *Proc. IS&T/SPIE Int. Symp. Electronic Imaging: Science and Technology*, vol. 1905, San Jose, CA, 1993, pp. 861–870.
- [2] UCI Machine Learning Repository, “Breast Cancer Wisconsin (Diagnostic) Data Set,” [Online]. Available: [https://archive.ics.uci.edu/ml/datasets/Breast+Cancer+Wisconsin+\(Diagnostic\)](https://archive.ics.uci.edu/ml/datasets/Breast+Cancer+Wisconsin+(Diagnostic)). [Accessed: Sep. 2026].
- [3] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [4] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, New York, NY, USA, 2016, pp. 785–794.
- [5] I. T. Jolliffe and J. Cadima, “Principal component analysis: A review and recent developments,” *Philos. Trans. Roy. Soc. A*, vol. 374, no. 2065, p. 20150202, 2016.